DermaUH versión 3.0: Integración de nuevos modelos para procesamiento de imágenes entrenados con imágenes de dermatólogos cubanos

José Miguel Leyva De la Cruz

19 de abril de 2026

Índice general

1. Introducción **2**
1.1. Motivación y Justificación 3
1.2. Antecedentes 3
1.3. Formulación Del Problema 4
1.4. Objetivos Generales y Específicos 5
1.5. Justificación 5
1.6. Estructura del Trabajo 6

2. Estado del Arte **7**

3. Solución Propuesta **8**
3.1. Metodología 8
3.2. Levantamiento de Requerimientos 10
3.2.1. Requerimientos Funcionales 10
3.2.2. Requerimientos No Funcionales 11
3.3. Análisis y Diseño 11
3.3.1. Diagrama de Casos de Uso 12
3.3.2. Propuesta Arquitectura 15
3.3.3. Modelado de la Base de Datos 17

4. Por donde voy **20**

Capítulo 1

Introducción

Desde tiempos inmemorables se ha buscado una forma de vincular las ciencias médicas con el desarrollo tecnológico, mejorando o creando herramientas digitales que permitan mejores resultados y ayuden incluso a doctores con gran experiencia. La dermatología, rama encargada de estudiar enfermedades producidas sobre la superficie de la piel, es una interesante propuesta para llevar a cabo esta conexión técnico-medicinal. El dermatólogo debe dar seguimiento a las lesiones cutáneas de sus pacientes, de esa forma si estos poseen una enfermedad grave, será diagnosticada a tiempo, elevando las posibilidades de cura o evolución positiva.

Para este seguimiento, una herramienta fundamental es el dermatoscopio, un instrumento óptico que permite visualizar estructuras de la piel no visibles a simple vista, ampliando las imágenes y eliminando el reflejo de la luz superficial. Las imágenes dermatoscópicas resultantes son el insumo principal para el análisis de lesiones pigmentadas y no pigmentadas, ya que revelan patrones y características morfológicas clave (como atipias, redes pigmentarias o vasos anormales) que orientan al especialista hacia un diagnóstico más certero. De esta forma, la combinación del dermatoscopio con el registro digital de imágenes ha abierto la puerta a soluciones automatizadas que asisten al médico en la detección temprana de patologías como el melanoma.

Según los autores del libro de referencia en la materia, Stuart Russell y Peter Norvig, la Inteligencia Artificial se define como el estudio de agentes computacionales que operan de forma autónoma, perciben su entorno, se adaptan al cambio y persiguen objetivos [6] Como un subcampo de esta, el Machine Learning (o aprendizaje automático) es el estudio de algoritmos que permiten a los programas informáticos mejorar automáticamente su rendimiento a través de la experiencia [8]. En esencia, el aprendizaje automático dota a los sistemas de la capacidad de aprender sin ser programados explícitamente para cada tarea.

Con el desarrollo y crecimiento de la Inteligencia Artificial (IA), específicamente el Aprendizaje de Máquinas (ML, por sus siglas en inglés) se han desarrollado diversas herramientas que ayudan no solo a dermatólogos, sino al personal de salud en general, en

sus diagnósticos a pacientes. Hoy en día el procesamiento de imágenes ha abierto nuevas puertas que permiten a los médicos dar un seguimiento más sencillo y cómodo de las distintas lesiones, disminuyendo la fatiga visual y el uso de equipos externos, además de ser usado para adiestramiento de futuros doctores.

1.1. Motivación y Justificación

La medicina tiene vital importancia para la sociedad, por ese motivo, lograr que los médicos tengan una herramienta que les facilite su trabajo, además de ser un reto para cualquier persona apasionada por la tecnología, constituye una motivación tanto social como humana. En el contexto cubano, es de gran ayuda informatizar la mayor cantidad de procesos, debido a las dificultades logísticas que muchas veces se presentan. Dotar al personal de la salud de una aplicación web que les permita desde la comodidad de su teléfono dar seguimiento a sus pacientes y verificar si presentan alguna lesión grave con tan solo una fotografía, provocaría un gran avance en el sector medicinal de la nación. Esa herramienta ya existe desde hace varios años, concretamente desde 2023 con el desarrollo del sitio web DermaUH, el cual era capaz de suplir todas estas necesidades mencionadas y brindar las comodidades planteadas. Sin embargo, con el alto crecimiento que han tenido las herramientas de IA y los modelos de ML, acompañado de la falta de un buen conjunto de imágenes de pieles cubanas para entrenar el modelo desarrollado en su momento, se hace necesario desarrollar una nueva versión de DermaUH y vincularle un nuevo modelo más potente con altos porcentajes de efectividad clasificando imágenes de pieles cubanas.

1.2. Antecedentes

El ecosistema DermaUH ha evolucionado a través de dos hitos fundamentales previos a la presente versión 3.0.

Versión 2.0: Sitio web DermaUH (Mauricio Salim Mahmud Sánchez, 2024) La versión 2.0 consistió en una plataforma web de apoyo al diagnóstico dermatoscópico, desarrollada como una aplicación de una sola página (SPA) con arquitectura cliente-servidor. El frontend se construyó con Vue.js y TypeScript, mientras que el backend implementó una API RESTful construida con Django y Django Rest Framework, utilizando PostgreSQL como sistema gestor de base de datos. La plataforma permite a los médicos registrarse (con verificación por administradores), gestionar pacientes (CRUD completo), registrar lesiones cutáneas asociadas a cada paciente, y almacenar imágenes clínicas y dermatoscópicas por cada lesión. Entre sus funcionalidades avanzadas se incluyen: clasificación automática de lesiones mediante un ensemble de redes neuronales convolucionales (previamente desarrollado por Claudia Olavarrieta), segmentación de imágenes mediante arquitectura U-Net

(desarrollada por Marcos Valdivié), y un sistema de colaboración que permite compartir lesiones entre especialistas para recibir diagnósticos adicionales. La plataforma también expone una API para fines investigativos, accesible por usuarios con permisos especiales. Todas las herramientas utilizadas son de código abierto. Esta versión sentó las bases del ecosistema, pero dependía completamente de conectividad a internet y estaba diseñada exclusivamente para uso en navegador web.

Versión móvil: Aplicación offline con sincronización eventual (Lázaro David Alba Ajete, 2025) Posteriormente, se desarrolló una aplicación móvil nativa (Android) que extiende el ecosistema al entorno offline. Implementada con Flutter y Dart, sigue los principios de Clean Architecture y el patrón MVVM. Utiliza Isar DB como base de datos local embebida y Riverpod para la gestión de estado. La aplicación permite a los dermatólogos, en ausencia de conexión a internet, registrar pacientes, capturar imágenes de lesiones (clínicas y dermatoscópicas), y ejecutar un modelo de clasificación de lesiones (Melanoma, Carcinoma Basocelular, Carcinoma Escamoso) mediante TensorFlow Lite embebido en el dispositivo. Los datos se almacenan localmente con campos de sincronización (`synced`, `deleted`, `remoteId`) y se transmiten de forma unidireccional al servidor central de DermaUH cuando se restablece la conectividad, mediante la API REST preexistente. La sincronización puede ser manual o automática diaria configurable. Esta versión resolvió el problema de la dependencia de internet, pero su modelo de IA es únicamente clasificador (sin capacidades explicativas) y el procesamiento de imágenes se limita a la inferencia del modelo previamente entrenado.

1.3. Formulación Del Problema

DermaUH pretende ser una herramienta potente de apoyo a especialistas en dermatología para el diagnóstico de las lesiones más frecuentes de cáncer de la piel. La versión 2.0, que actualmente corre, ha sido evaluada y se han detectado varios problemas que implicarán desarrollar nuevamente el frontend y el backend de la misma. Además el modelo actual de diagnóstico, que es un ensemble de 3 redes neuronales no fue entrenado con imágenes cubanas. Por este motivo se desea desarrollar una nueva versión del sitio DermaUH, la cual tendrá como principal mejora, la incorporación de nuevos modelos de procesamiento de imágenes. A continuación se expondrán las limitantes y/o deficiencias detectadas en las versiones anteriores del sitio web DermaUH.

1. El modelo de ML utilizado para clasificar imágenes no se entrenó con imágenes de pieles cubanas, alejándose en ese aspecto del cliente objetivo, los dermatólogos cubanos.

2. El diagnóstico asistido no viene acompañado de una justificación que responda a la interrogante ¿Por qué se obtuvo este resultado?.

3. El frontend de la aplicación tarda alrededor de diez segundos en cargar todos los elementos, haciendo lenta y molesta la experiencia del usuario.

Por ende el problema de este trabajo puede ser enunciado como la carencia de algunas funcionalidades que presenta la aplicación anterior. Dando paso a la siguiente pregunta científica **¿Será posible dotar a la aplicación web DermaUH de una actualización que supere todas las limitantes antes planteadas y continue cumpliendo con todos los requerimientos que se expondrán en este documento?**

1.4. Objetivos Generales y Específicos

Como objetivo general se tendrá desarrollar la versión 3.0 de DermaUH, la cual tendrá un nuevo modelo de procesamiento de imágenes entrenado con imágenes de pieles cubanas, mejorará el rendimiento de la aplicación tanto del frontend como el backend, mejorará el protocolo de seguridad y manejo de solicitudes y notificaciones mediante un cliente de correo Email. Además esta versión será validad en 2026 en ensayo clínico. De aquí se derivan varios objetivos específicos:

1. Analizar herramientas existentes para el trabajo con imágenes dermatoscópicas así como sus ventajas y desventajas para el contexto cubano.

2. Lograr el desarrollo del frontend y backend que supere las deficiencias presentadas en la v2.0.

3. Mejorar las métricas de los modelos desarrollados e incorporarlos a la app una vez que sean reentrenados con el nuevo conjunto de imágenes de pieles cubanas.

4. Lograr una app web y explique al especialista la decisión del diagnóstico mediante el modelo explicativo (XAI) desarrollado por la Lic.Amanda Noris.

5. Intercambiar con los especialistas en cada etapa del trabajo

1.5. Justificación

Se ceunta con una alta capacidad para el desarrollo de la versión 3.0 de DermaUH, debido a todos los conocimientos que se sienten sobre el stack tecnológico seleccionado y los adquiridos durante la carrera de Ciencias de la Computación. Además de eso existe abundante bibiliografía sobre los temas abordados (IA, ML, Procesamiento de imágenes), lo cual facilita mucho la búsqueda de información para el desarrollo. Por último, la facultad

MatCom ya tiene experiencia en el desarrollo de aplicaciones con un alto componente de IA, sobre todo en aquellas que su principal cualidad está vinculada al procesamiento de imágenes.

1.6. Estructura del Trabajo

Capítulo 2

Estado del Arte

Capítulo 3

Solución Propuesta

Como se explicó en el capítulo anterior, existen numerosas aplicaciones y sitios webs con un enfoque similar a DermaUH, todos por supuesto con sus ventajas y desventajas para el cliente objetivo, los dermatólogos cubanos. La competencia, la exigencia, el tiempo del que se dispone para desarrollo, provoca una planificación milimétrica si se desean cumplir con todas los requerimientos y objetivos planteados al inicio del documento. El presente capítulo explicará desde el punto de vista metodológico y analítico la solución propuesta de DermaUH 3.0, haciendo enfásis en la Metodología, los Requerimientos y el Diseño.

3.1. Metodología

Una metodología de desarrollo de software es un marco de trabajo que se usa para estructurar, planificar y controlar el proceso de desarrollo de sistemas de información. Las metodologías de Desarrollo de Software (DS.) han experimentado un proceso histórico y evolutivo que inicia en los años 40 con la aparición de las primeras computadoras. Hoy en día existen dos tipos de metodologías, tradicionales y ágiles. [4]

1- **Metodologías Tradicionales:** Centran su atención en llevar una documentación exhaustiva de todo el proyecto, planificación y control del mismo. Imponen una disciplina rigurosa, con el fin de lograr un software más eficiente. Algunas de estas metodologías son:

1.1- **Cascada:** Consiste en fases secuenciales donde cada una debe completarse antes de pasar a la fase siguiente. Es muy fácil de entender y gestionar, es ideal cuando los requisitos son estables y bien conocidos, sin embargo es muy rígida antes cambios y el cliente solo ve el producto final.

1.2- **V-Model:** Es una variante del Cascada que enfatiza la verificación y validación (V&V), cada fase de desarrollo tiene una fase de prueba correspondiente. Presenta una alta calidad y trazabilidad, además las pruebas se planifican desde el

inicio. Por otro lado sigue siendo rígido como Cascada y no es apto para requisitos cambiantes.

1.3- **Modelo Espiral:** Propuesto por Barry Boehm, combina Cascada con gestión de riesgos mediante iteraciones que aumentan en detalle. Este enfoque permite iteraciones controladas y es bueno para proyectos grandes con incertidumbre técnica, sin embargo es complejo de gestionar y puede ser costoso en cuanto a recursos.

2- **Metodologías Ágiles:** Generalmente son procesos *Incrementales* (entregas en ciclos cortos y rápidos), *Cooperativo* (clientes y desarrolladores trabajan constantemente con una comunicación muy fina y constante), Sencillo (el método es fácil de aprender y modificar para el equipo) y finalmente Adaptativo (capaz de permitir cambios de último momento). Estas metodologías ponen de relevancia que la capacidad de respuesta a un cambio es más importante que el seguimiento estricto de un plan [7]. Entre las más conocidas se pueden apreciar:

2.1- **Scrum:** Se basa en ciclos de trabajos cortos llamados *Sprints* que tienen una duración de entre 2 y 4 semanas. Es ideal para proyectos complejos que requieren flexibilidad y entregas rápidas. [5]

2.2- **Kanban:** Kanban se centra en visualizar el flujo de trabajo y limitar el trabajo en progreso (WIP, por sus siglas en inglés). No utiliza iteraciones de tiempo fijo como Scrum, sino que se basa en un flujo continuo de tareas. Es ideal para equipos que buscan mejorar la eficiencia y la gestión de tareas, pero puede ser menos estructurado que Scrum.

2.3- **Extreme Programming (XP):** XP se enfoca en las prácticas de ingeniería de software para producir código de alta calidad y adaptarse a requisitos cambiantes. Es especialmente adecuado para proyectos con requisitos muy volátiles. Promueve prácticas como la programación en parejas (dos desarrolladores trabajan en el mismo código), el desarrollo guiado por pruebas (TDD), la integración continua y las refactorizaciones frecuentes para mejorar el diseño del código.

Para el desarrollo de DermaUH 3.0 se ha decidido utilizar la metodología Scrum, debido a que es una metodología ágil que se adapta bien a proyectos complejos y con requisitos cambiantes, como es el caso de DermaUH. Además, Scrum permite entregas rápidas y frecuentes, lo que facilita la retroalimentación constante del cliente y la adaptación a sus necesidades. Para ello se dividió el proceso de desarrollo en X sprints, con una duración promedio de entre dos y tres semanas cada uno. Al final de cada sprint se realizó una evaluación y validación de los resultados obtenidos. Los sprints en los que fue dividido el proceso son los siguientes:

1. Desarrollo de toda la sección de Registro e Inicio de Sesión de los usuarios. Definir roles y permisos de cada uno.

2. Implementar un servicio de correo, el cual notifica al usuario administrador de nuevas solicitudes y a los usuarios les notifica la respuesta de su solicitud.

3. Desarrollar las funcionalidades básicas de la Aplicación, gestión pacientes, lesiones y diagnósticos.

4. Entrenar los nuevos modelos a integrar con el nuevo conjunto imágenes entregado por las organizaciones de la salud.

5. Desarrollar el servicio de consulta de Imagen y vincular el modelo a la aplicación DermaUH 3.0

6. Realizar pruebas de rendimiento y ejecución con doctores para posteriormente lanzar la aplicación.

3.2. Levantamiento de Requerimientos

El levantamiento de requerimientos es el proceso mediante el cual se identifican, documentan y acuerdan las necesidades, expectativas y restricciones que debe cumplir un sistema de software. Incluye tanto los requerimientos funcionales (qué debe hacer el sistema) como los no funcionales (cómo debe hacerlo: rendimiento, seguridad, usabilidad, etc.). Este proceso involucra a los stakeholders (clientes, usuarios finales, desarrolladores) y constituye la base sobre la que se construye todo el proyecto.

Los requerimientos son importantes porque actúan como el contrato implícito entre el equipo de desarrollo y los interesados. Unos requerimientos claros y bien gestionados permiten: definir el alcance realista del proyecto, evitar malentendidos y retrabajos costosos, establecer criterios objetivos de validación y aceptación y priorizar funcionalidades según el valor que aportan. La falta de una buena gestión de requerimientos es una de las causas más frecuentes de fracaso en proyectos de software, ya que conduce a entregas que no satisfacen las necesidades reales, sobrecostes y demoras.

3.2.1. Requerimientos Funcionales

Según un artículo de Michel Arias Chaves, para la revista InterSede [3] los requerimientos funcionales son los que definen las funciones que el sistema será capaz de realizar, describen las transformaciones que el sistema realiza sobre las entradas para producir salidas.Es importante que se describa el ¿Qué? y no el ¿Cómo? se deben hacer esas transformaciones. Estos requerimientos al tiempo que avanza el proyecto de software se convierten en los algoritmos, la lógica y gran parte del código del sistema. Siguiendo este concepto, los requerimientos funcionales de DermaUH 3.0 son:

RF1 El usuario autenticado puede hacer **CRUD** sobre las entidades Paciente, Lesión y diagnóstico.

RF2 Los usuarios podrán obtener información de lesiones registradas, solo de sus propios pacientes, podrán acceder a otras lesiones si estas se hacen públicas o se comparten con el mismo.

RF3 El usuario puede enviar una imagen al sistema y recibir un diagnóstico de la misma.

RF4 Solamante el administrador tiene permiso para eliminar cuentas y activar usuarios en el sistema y consultar detalles de cualquier cuenta.

RF5 Cualquier persona puede enviar una solicitud de registro a la aplicación, la cual después será aprobada por el Administrador.

3.2.2. Requerimientos No Funcionales

los requerimientos no funcionales tienen que ver con características que de una u otra forma puedan limitar el sistema, como por ejemplo, el rendimiento (en tiempo y espacio), interfaces de usuario, fiabilidad (robustez del sistema, disponibilidad de equipo), mantenimiento, seguridad, portabilidad, estándares, etc. Siguiendo esta línea se concluye que los requerimientos no funcionales de DermaUH son:

RNF1 La autenticación debe ser mediante email y password generando un JSON Web Token (JWT) el cual definirá los permisos del usuario que hace inicio de sesión. El JWT a su vez debe tener un tiempo de expiración y ser renovable

RNF2 La comunicación entre cliente y servidor debe estar cifrada mediante HTTPS.

RNF3 El tiempo de operación de una petición y la generación de la respuesta debe ser inferior a dos segundos con internet estable.

RNF4 La interfaz web debe ser responsive (adaptable para móviles, tabletas y ordenadores).

RNF5 Cualquier respuesta a una solicitud, sea positiva o de error debe mostrarse con mensajes claros a los usuarios.

3.3. Análisis y Diseño

Una vez definidos los requisitos funcionales y no funcionales que debe cumplir el sistema, el siguiente paso consiste en modelar su estructura y comportamiento desde una perspectiva técnica. Este capítulo presenta los artefactos de diseño que guiarán la implementación de la versión 3.0 de DermaUH. Se abordarán temas como: casos de uso

que aparecen en la aplicacón, arquitectura de software seleccionada así como estructura, diseño y modelación de las entidades de la base de datos.

3.3.1. Diagrama de Casos de Uso

Los diagramas de casos de uso se utilizan durante la fase de análisis de un proyecto para identificar y dividir la funcionalidad del sistema. Normalmente contienen: casos de uso, actores y relaciones entre ellos: de asociación, de dependencia y/o de generalización [9]. En otras palabras es una forma de esquematizar y representar los requerimientos funcionales planteados en el epígrafe anterior.

- **Casos de Uso:** Los casos de uso describen el comportamiento del sistema cuando uno de los actores envía un estímulo concreto.
- **Actores:** Los actores interpretan papeles que pueden ser representados por los usuarios del sistema. Dichos usuarios pueden ser humanos, otros ordenadores o incluso otros sistemas de software

A continuación se mostrarán los diagramas de casos de uso, realizados con la herramienta diagrams.net debido a que ofrece una gran variedad de opciones de forma gratuita. Normalmente en un diagrama de casos de uso se visualizan todos los actores y los casos de uso, por simplicidad y para que sea más entendible para el lector, se separaron los diagramas por actores.

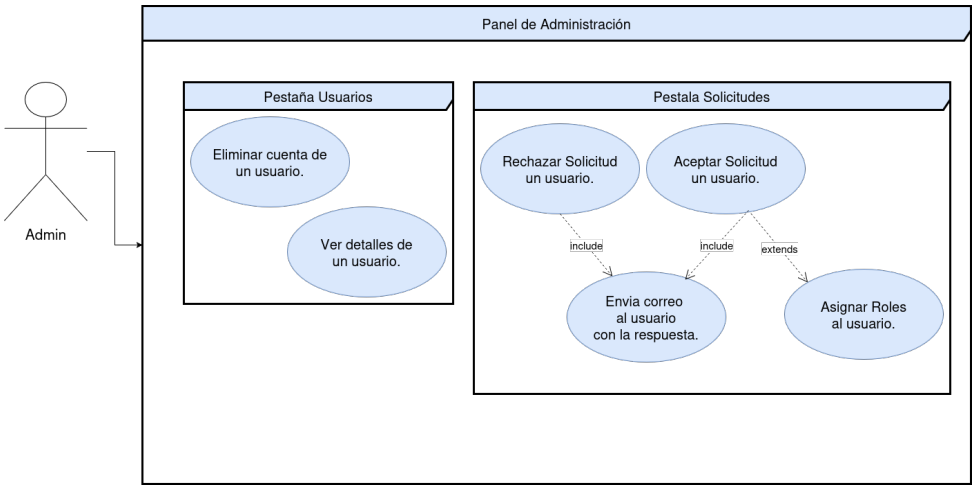

Figura 3.1: Diagrama UML del actor Admin

La figura 3.1 muestra las acciones permitidas cuando el usuario entra al sistema con permisos de administrador. A continuación una breve explicación de cada una:

En la pestaña *Usuarios*:

- **Eliminar Cuenta del Usuario:** Esta opción permite eliminar la cuenta de un usuario de la aplicación, sin importar los permisos del usuario ni el tiempo que lleve activo en el sistema.

- **Ver Detalles del Usuario:** Esta opción le muestra al administrador todos los datos personales que el usuario ingresó en el formulario de registro, como su nombre completo, hospital al que pertenece, correo entre otros. En caso del usuario ingresar como residente, muestra también el correo del doctor encargado de tutorearlo.

En la pestaña *Solicitudes*:

- **Rechazar Solicitud de Usuario:** Con esta opción el administrador rechaza, lo cual es sinónimo de eliminar, al menos en este contexto, la solicitud de registro enviada por un usuario.

- **Aceptar Solicitud de Usuario:** Permite al administrador aceptar la solicitud de registro de un usuario y este ya podrá utilizar la aplicación en dependencia de los permisos asignados por el administrador.

- **Asignar Roles al Usuario:** Una vez aceptada la solicitud de registro, el adminsitrador debe asignarle permisos a dicho usuario. Los permisos permitidos son administrador y/o doctor.

- **Enviar correo:** Esta acción se dispara automáticamente cuando se acepta o rechaza una solicitud de registro de un usuario. En caso de rechazo se le explicará las posibles causas del mismo, así como todos los datos ingresados por el usuario durante su registro, sin incluir la contraseña.

Evidentemente el usuario con permisos de Administrador es el único que tiene poder y potestad para manejar todo lo relacionados con las cuentas de los usuarios y las solicitudes de registro. A continuación se mostrará que ocurre en caso del usuario tener permisos de doctor

- **Registrar Nuevos Pacientes:** En el panel principal de la aplicación, arriba a la derecha, aparece un botón para registrar pacientes nuevos en el sistema. Al presionarlo se muestra un formulario que debe ser rellenado con información personal del paciente, nombre, edad, sexo, profesión etc. Una vez rellenado el formulario, se presiona el botón de finalizar registro y el paciente queda registrado en el sistema.

- **Ver detalles de un Paciente:** Los pacientes son representados como tarjetas en el panel principal. A la derecha de cada tarjeta se muestra un botón con la etiqueta *Detalles* que permite al doctor visualizar los datos personales del paciente, separada en tres secciones, información personal, información étnica y dirección.

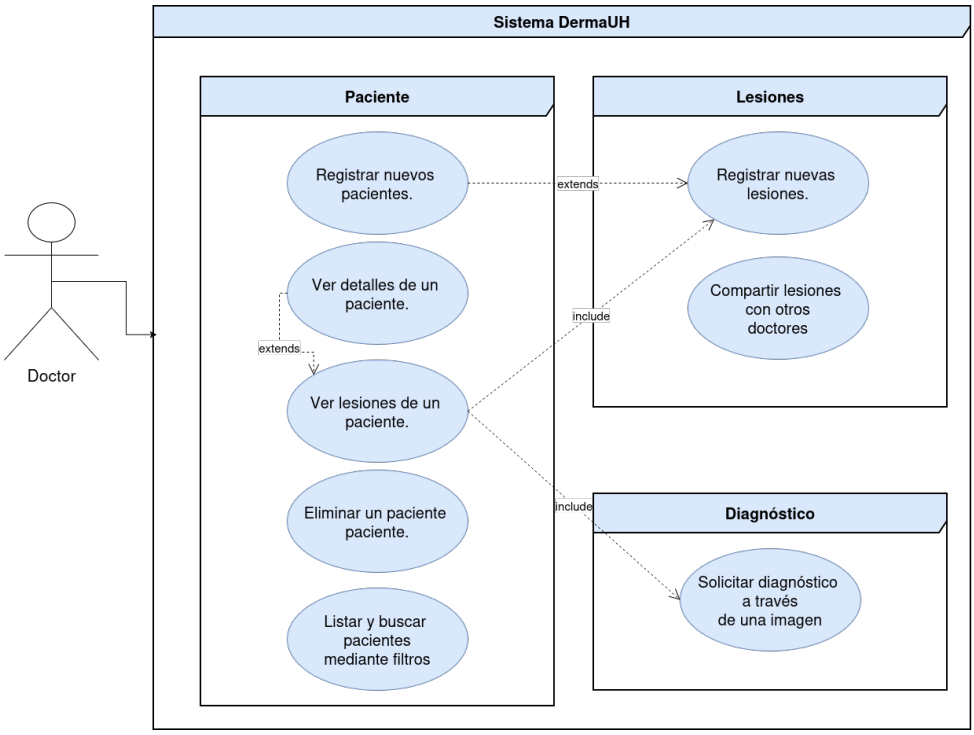

Figura 3.2: Diagrama UML del actor Doctor

- **Ver lesiones de un Paciente:** Similar a la opción anterior, a la derecha del paciente se muestra el botón *Lesiones*. Este le permite al doctor observar el registro de lesiones del paciente en cuestión.

- **Registrar nuevas lesiones:** Se puede llegar a esta opcioón de dos formas diferentes, cuando se registra un nuevo paciente y se presiona el botón finalizar registro, aparece un nuevo formulario para registrar la neuva lesión y la otra es al acceder al listado de lesiones de un paciente se muestra un botón para agregar una lesión nueva.

- **Eliminar un Paciente:** Similar a ver lesiones de un paciente y ver detalles se accede a esta opción mediante un botón ubicado a la derecha de la tarjeta del paciente.

- **Listar y buscar pacientes mediante filtros:** En la parte superior del panel principal aparece una barra de búsqueda, donde el doctor podrá introducir una consulta y la aplicación filtrará a los pacientes por la consulta del doctor. Además aparece una opción para ordenar los pacientes por algún campo.

- **Compartir lesiones con otros doctores:** Con esta opción, el doctor tiene la posibilidad de compartir una lesión con otros colegas del personal de la salud. Los doctores escogidos para compartir tendrán la posibilidad de ver solamente el nombre y apellido del paciente y las imágenes de la lesión.

- **Solicitar diagnóstico a través de la imagen:** Esta opción permite al doctor tomar una fotografía, ya sea con el móvil o con el dermatoscopio y solicitar un diag-

nóstico aproximado de confianza proporcionado por un modelo de ML especializado en procesamiento de imágenes.

3.3.2. Propuesta Arquitectura

Como se mencionó al inicio del documento uno de los objetivos de esta tesis era el desarrollo desde cero tanto del frontend como el backend de la aplicación. Debido al stack tecnológico seleccionado (el cual se explicará en el próximo capítulo) se optó por una arquitectura por capas. En este epígrafe se explicará con más detalle esta arquitectura y se visualizará un diagrama de la misma.

La arquitectura en capas consta en dividir la aplicación en capas, con la intención de que cada capa tenga un rol muy definido, como podría ser, una capa de presentación (UI), una capa de reglas de negocio (servicios) y una capa de acceso a datos (DAO). En la práctica, la mayoría de las veces este estilo arquitectónico es implementado en 4 capas, presentación, negocio, persistencia y base de datos, sin embargo, es habitual ver que la capa de negocio y persistencia se combinan en una solo capa, sobre todo cuando la lógica de persistencia está incrustada dentro de la capa de negocio, como es el caso de DermaUH. En una arquitectura en capas, todas las capas se colocan de forma horizontal, de tal forma que cada capa solo puede comunicarse con la capa que está inmediatamente por debajo, por lo que, si una capa quiere comunicarse con otras que están mucho más abajo, tendrán que hacerlo mediante la capa que está inmediatamente por debajo [2].

La separación de la aplicación en capas busca cumplir con el principio de separación de preocupaciones, de tal forma que cada capa se encargue una tarea muy definida, por ejemplo, la capa de presentación solo se preocupa por presentar la información de forma agradable al usuario, pero no le interesa de donde viene la información ni la lógica de negocio que hay detrás, en su lugar, solo sabe que existe una capa de negocio que le proporcionará la información. Por otra parte, la capa de negocio solo se encarga de aplicar todas las reglas de negocio y validaciones, pero no le interesa como recuperar los datos, guardarlos o borrarlos, ya que para eso tiene una capa de persistencia que se encarga de eso. Por otro lado, la capa de persistencia es la encargada de comunicarse a la base de datos, crear las instrucciones SQL para consultar, insertar, actualizar o borrar registros y retornarlos en un formato independiente a la base de datos. De esta forma, cada capa se preocupa por una cosa y no le interesa como le haga la capa de abajo para servirle los datos que requiere. A continuación se muestra un resumen de las ventajas de optar por una arquitectura por capas.

- **Separación de responsabilidades:** Permite la separación de preocupaciones (SoC), ya que cada capa tiene una sola responsabilidad.

- **Fácil de desarrollar:** Este estilo arquitectónico es especialmente fácil de implementar, además de que es muy conocido y una gran mayoría de las aplicaciones la

utilizan.

- **Fácil de probar:** Debido a que la aplicación construida por capas, es posible ir probando de forma individual cada capa, lo que permite probar por separada cada capa.
- **Fácil de mantener:** Debido a que cada capa hace una tarea muy específica, es fácil detectar el origen de un error para corregirlo, o simplemente se puede identificar donde se debe aplicar un cambio.
- **Seguridad:** La separación de capas permite el aislamiento de los servidores en subredes diferentes, lo que hace más difícil realizar ataques.

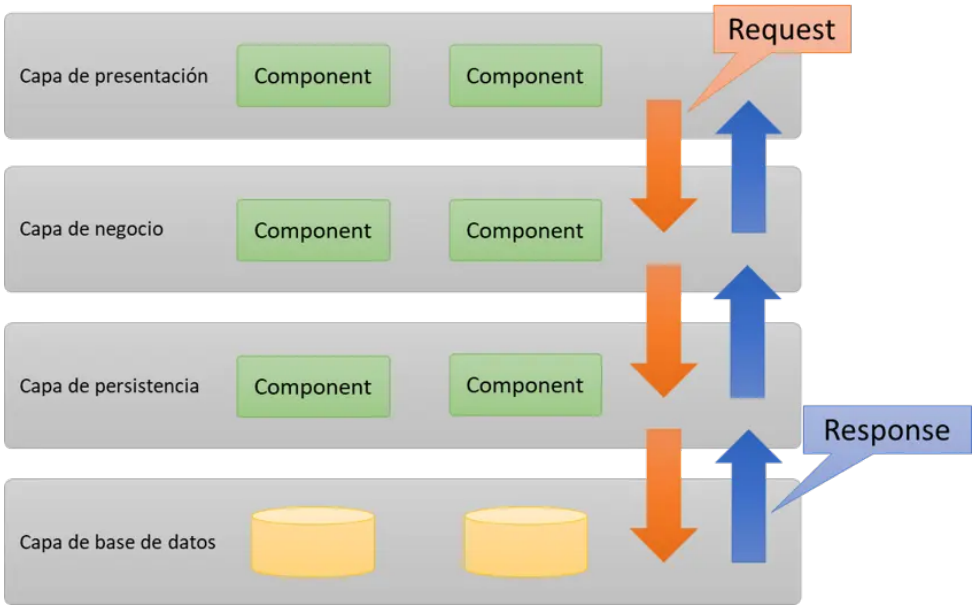

Figura 3.3: Arquitectura basada en capas

Una vez analizadas las ventajas de esta arquitectura, el stack tecnológico utilizado y la experiencia en el desarrollo de aplicaciones con esta arquitectura, se llegó a la conclusión de que es la mejor propuesta para llevar a cabo el desarrollo de la versión 3.0 de la aplicación DermaUH. Para ello se decidió utilizar tres capas: la primera será la de presentación (UI), en esta estará todo el desarrollo del frontend y será la encargada de hablar con el usuario; la segunda capa es la lógica de negocio y persistencia, en esta estará todo lo relacionado con el backend de la aplicación y la comunicación con la Base de Datos (BD), será el puente entre la UI y la BD; finalmente, la tercera capa, que es la BD, en ella se almacenarán todos los datos de doctores, pacientes, lesiones y será el núcleo de la generación de respuestas a peticiones del usuario.

3.3.3. Modelado de la Base de Datos

Definida la arquitectura del sistema, el siguiente paso consiste en estructurar los datos que persistirán y las relaciones entre ellos. El presente apartado describe el modelo de datos de DermaUH 3.0, incluyendo las entidades principales (usuario, paciente, lesión, imagen, diagnóstico automático, entre otras), sus atributos, las claves primarias y foráneas, y las asociaciones que garantizan la integridad y coherencia de la información. Se presenta el Modelo Entidad-Relación, también conocido como Diagrama de Entidad Relación (DER).

Modelo de Entidad Relación (DER)

Según un artículo obtenido del sitio web UNIR, herramienta elaborada, desarrollada y diseñada con el propósito de ayudar a los estudiantes de universidades europeas, El modelo entidad relación (ERD o modelos ER) es una herramienta que permite representar de manera simplificada cómo personas, objetos o conceptos se relacionan entre sí. Se utiliza para exponer cómo se organiza la información en una base de datos [1]. La creación del modelo entidad relación para su aplicación en el diseño de bases de datos se le atribuye a Peter Chen, profesor del MIT, quien publicó en 1976 el documento "Modelo entidad-relación: hacia una visión unificada de los datos". A continuación se expone el DER utilizado para la modelación de la base de datos.

TODO: HACER EL DIAGRAMA CUANDO LA HERRAMIENTA LE DE LA GANA

Paciente

El modelo de datos, **Paciente**, es la entidad más importante, ya que es el núcleo de una de las principales funcionalidades de DermaUH 3.0. A continuación una descripción de los atributos de dicha entidad.

- idPaciente: Llave primaria (PK), identificador único de tipo entero. Es generado autoáticamente por el sistema de manera incremental.
- name, lastName1, lastName2: Campos para obtener el nombre completo del paciente. Todos son de tipo varchar, con una longitud máxima de cien caracteres por cada campo.
- yeras, sex, skinColor: Campos relacionados con la información étnica del paciente. Estos campos su principal funcionalidad es brindar más información para obtener un mejor diagnóstico asistido por parte de los modelos ML integrados en la nueva versión.
- addres, city, state, jobProfession: Campos para reflejar la dirección y profesión laboral del paciente. Todos son de tipo varchar sin límites. Estas propiedades son

importantes para saber el nivel exposición de la piel al sol, debido la heterogeneidad del clima en diferentes sitios del país.

- registerData: Campo de tipo Date para tener control de cuando fue que el paciente fue registrado por primera vez en el sistema de DermaUH.

Dermatólogo

Esta es una entidad fuerte, ya que no depende de otras entidades para instanciarse. El modelo **Dermatólogo**, es el responsable de representar la información de los usaurios del sistema. En pocas palabras, un dermatólogo es un usuario de DermaUH 3.0.

- idDermatologo: Llave primaria (PK), identificador único de tipo entero. Es generado autoáticamente por el sistema de manera incremental.
- name, lastName1, lastName2: Campos para obtener el nombre completo del dermatólogo. Todos son de tipo varchar, con una longitud máxima de cien caracteres por cada campo.
- email: Campo de tipo text, se usa para guardar el email del dermatólogo. En el sistema este campo es utilizado como username (nombre de usuario) y como el identificador para hacer inicio de sesión.
- ci: Carnet de Identidad del Usuario, campo de tipo varchar.
- phoneNumber: Campo para guardar el número telefónico del usuario.
- hospital, state, city: Campos de tipo de varchar para guardar el hospital, municipio y provincia respectivamente del usuario.
- doctorId: Identificador que se genera según el hospital, municipio y provincia del dermatólogo.
- userType: Campo de tipo varchar. Solo hay tres posibles valores para este campo: Admin, Doctor, Resident, es utilizado para manejar los permisos y roles de los usuarios de la aplicación.
- tutorEmail: Campo de tipo varchar, almacena el correo del tutor en caso del usuario ser residente, o estudiante de intercambio. En caso contrario este campo se puede mantener null.

Lesión

Entidad débil, modela el comportamiento de las lesiones que puede presentar un paciente. Un paciente puede presentar muchas una o muchas lesiones, por tanto el modelo **Lesión** depende de que exista un paciente. Se configuró con eliminación **Cascada**, por ende si un paciente es eliminado del sistema, se eliminarán todas sus lesiones asociadas.

- date: Fecha en la que se registra la lesión. Es un campo de tipo date, no editable.
- phototype: Campo de tipo entero, solo puede tomar valores en el intervalo $[1; 6]$ que se usa para almacenar el fototipo de la piel donde se encuentra ubicada la lesión. El fototipo no es más que la reacción de la piel ante la exposición a la luz solar.
- localizarion: Campo de tipo entero, toma valores en el intervalo $[1; 10]$ y almacena la parte del cuerpo donde se encuentra la lesión.
- histopathologicalDiagnosis: Campo de tipo bool. Indica si la lesión fue diagnosticada histopatólogicamente, en otras palabras, afirma que se utilizaron equipos médicos para obtener el diagnóstico.
- injuryType: Llave foránea (FK) que referencia a la entidad **Tipo de Lesión**. Es un campo de tipo entero.
- patientId: Llave foránea (FK) que referencia a la entidad **Paciente**. Es un campo de tipo entero.

Capítulo 4

Por donde voy

Bibliografía

[1] ¿qué es el modelo entidad relación y para que se utiliza? *UNIR: La Universidad en Internet*, I, 2024.

[2] Oscar Blancarte. Arquitectura en capas. *Software Architect*, 2021.

[3] Michel Arias Chaves. La ingeniería de requerimientos y su importancia en el desarrollo de proyectos de software. *InterSedes: Revista de las Sedes Regionales*, VI(10), 2005.

[4] Hellen Cubero. Procesos de ingeniería de software. *Universidad de San Marcos*, I, 2020.

[5] Claire Drumond. ¿qué es scrum? desglose del marco de trabajo ágil. *Director de Marketing y Producto*, I(1), 2025.

[6] Russell S. J. and P. Norvig. *Artificial Intelligence: A Modern Approach.* Pearson, 2021.

[7] Yura Luzhko. ¿qué son las metodologías ágiles? *Gerente de SEO*, I(1), 2026.

[8] T. M. Mitchell. *Machine Learning.* McGraw-Hill, 1997.

[9] Mari Carmen Otero Vidal. Teoría ingeniería de software. *Dpto. de Lenguajes y Sistemas Informáticos*, 2023.